BASIC LAB

子計畫一
Hardware-Friendly Highways: From Efficient Dynamic Routing to MoE Architectures

Presenter: 林奇毅

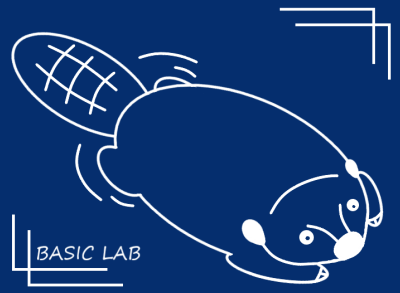
Advisor: 帥宏翰 教授

NYCU

Outline

- Introduction & Motivation
- Sparse Inference for LLMs
  - Dynamic Input Sparsity
  - Dynamic Width Sparsity
  - Dynamic Depth Sparsity
    - Pruning and Early Exit
    - Recurrence and Looped Architectures
    - Dynamic Routing and Modular Inference
- Proposed Research
  - Hardware-Friendly Highway
  - MoE Speculative Decoding

Introduction & Motivation



BASIC LAB

Introduction & Motivation

BASIC LAB

Bottleneck in LLM inference

- **Computational Intensity**: LLMs require massive FLOPs, leading to high deployment costs.
- **Memory-Bound Nature**: Inference is heavily restricted by memory bandwidth rather than raw compute power.
- **Utilization Gap**: hardware utilization remains low due to data movement overheads.

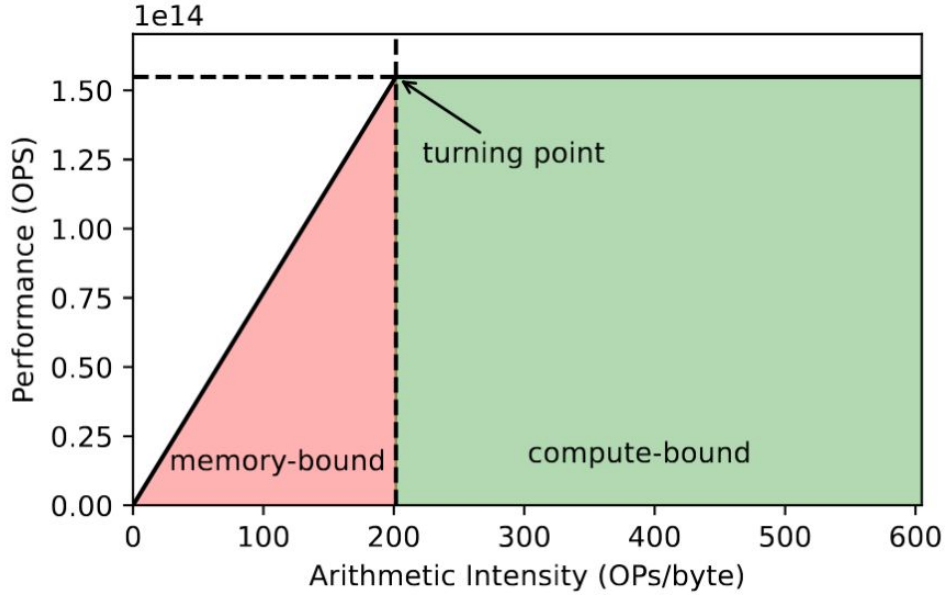


Figure 5. Demonstration of the Roofline model of Nvidia A6000 GPU. The computation is in FP16.

NYCU

Introduction & Motivation

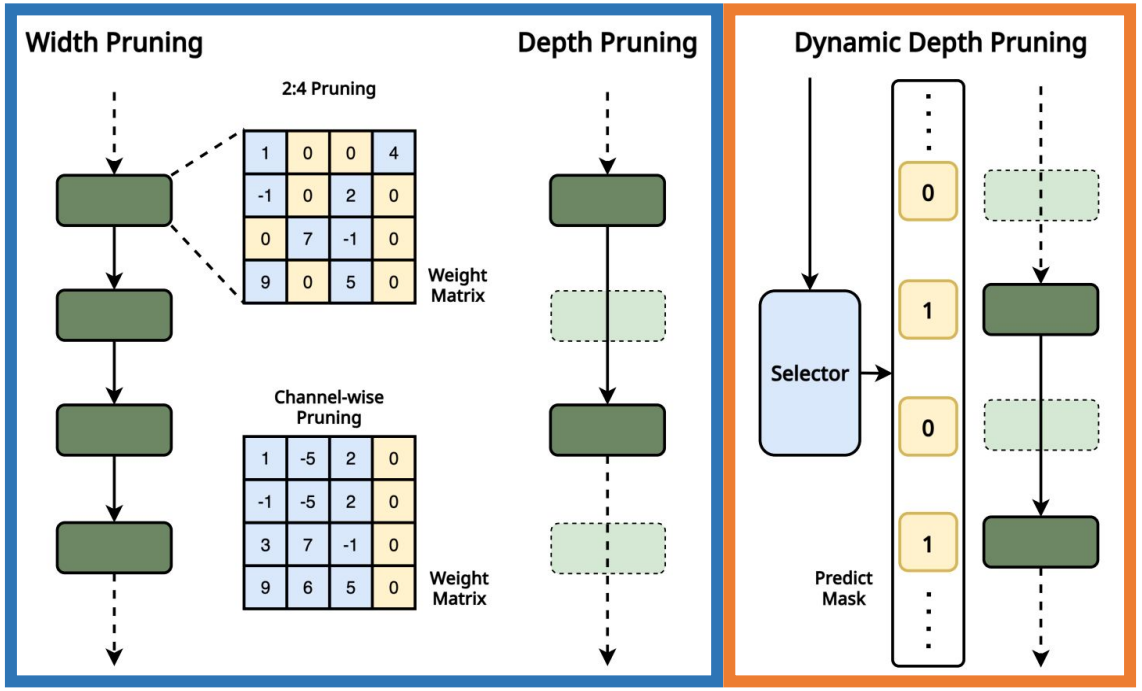
BASIC LAB

Static Pruning

- Removes redundant parameters permanently.
- **Pain Point**: Often leads to significant **accuracy degradation** and lacks input-dependent flexibility.

Dynamic Routing / Pruning

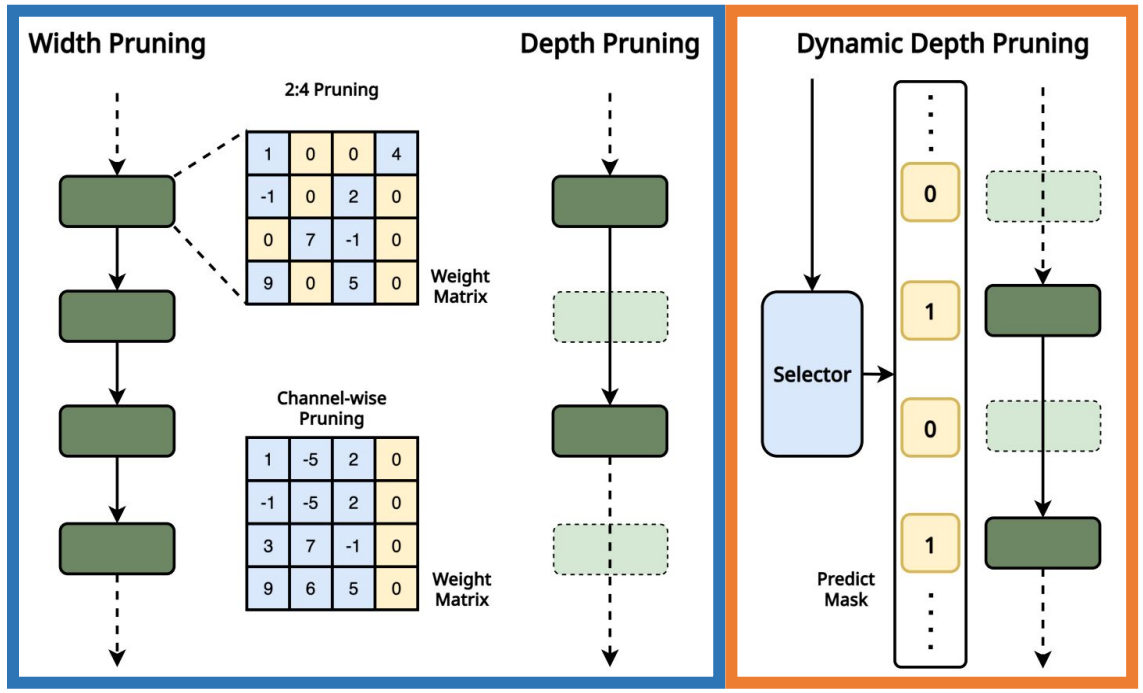
- Activates specific paths based on input complexity.
- **Pain Point**: Causes **branch prediction failures** and **non-contiguous memory access**, limiting actual speedups on standard hardware.



Common Routing Patterns
&
Hardware-friendly highways

Introduction & Motivation

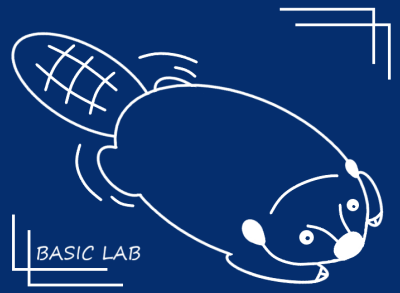
BASIC LAB



Common Routing Patterns
&
Hardware-friendly highways

- **Efficient Dynamic Routing**: Reducing depth-wise redundancy.
- **Scalable MoE Architectures**: Expanding width-wise capacity without increasing inference cost.

Sparse Inference for LLMs



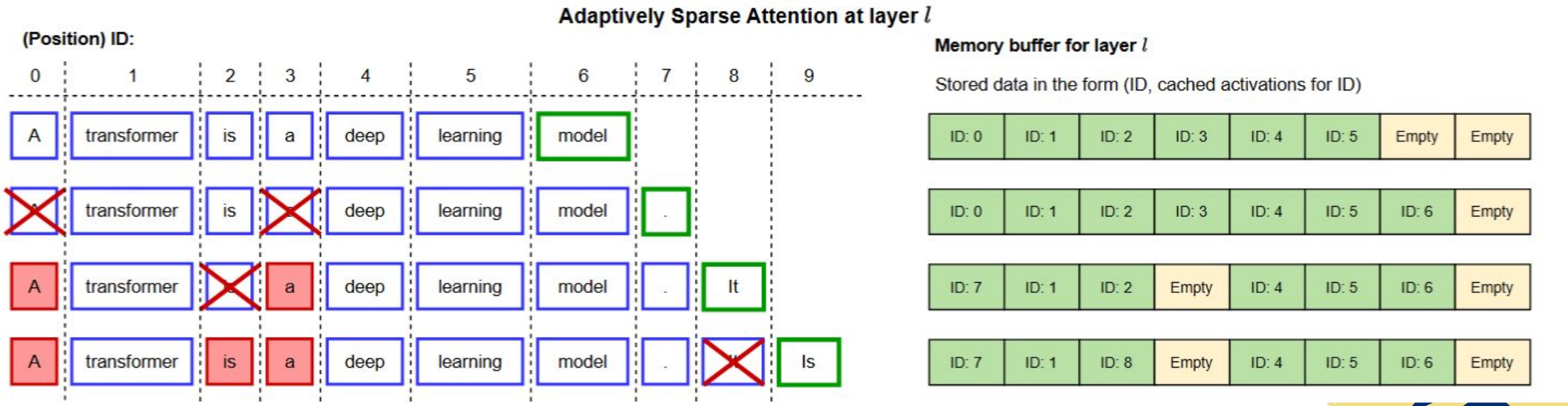
BASIC LAB

NYCU

Dynamic Input Sparsity

BASIC LAB

- Token Pruning
- Token Merging
- Dynamic KV Cache Eviction



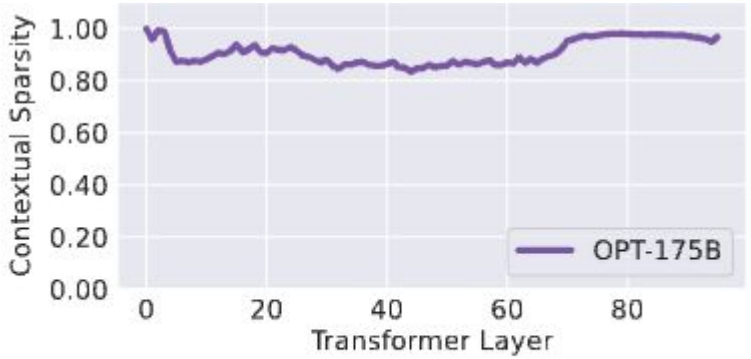
Dynamic Context Pruning for Efficient and Interpretable Autoregressive Transformers
https://arxiv.org/abs/2305.15805

Dynamic Input Sparsity

BASIC LAB

Why input sparsity: contextual sparsity exists and can be predicted.

85% contextual sparsity

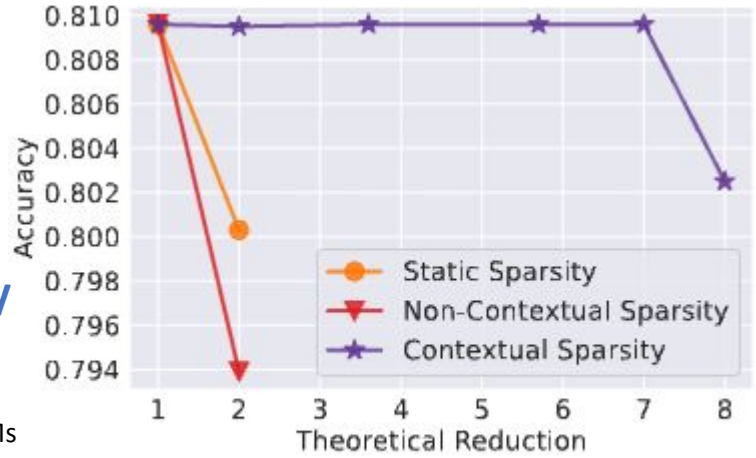


(a) Contextual Sparsity

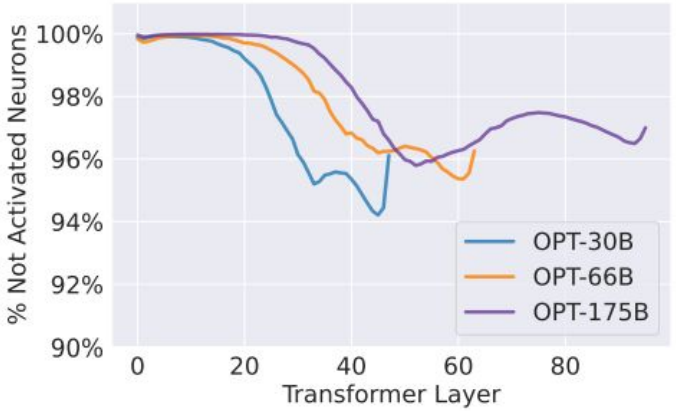
(a) Contextual sparsity in Attention Head

80% sparsity In Attention Head

Parameter Reduction without sacrificing accuracy



(b) Accuracy-Efficiency Trade-offs



(b) Contextual sparsity in MLP Block

95% sparsity in MLP Block

Deja Vu: Contextual Sparsity for Efficient LLMs at Inference Time
https://arxiv.org/abs/2310.17157

Deja Vu: Contextual Sparsity for Efficient LLMs at Inference Time

(PMLR 2023)

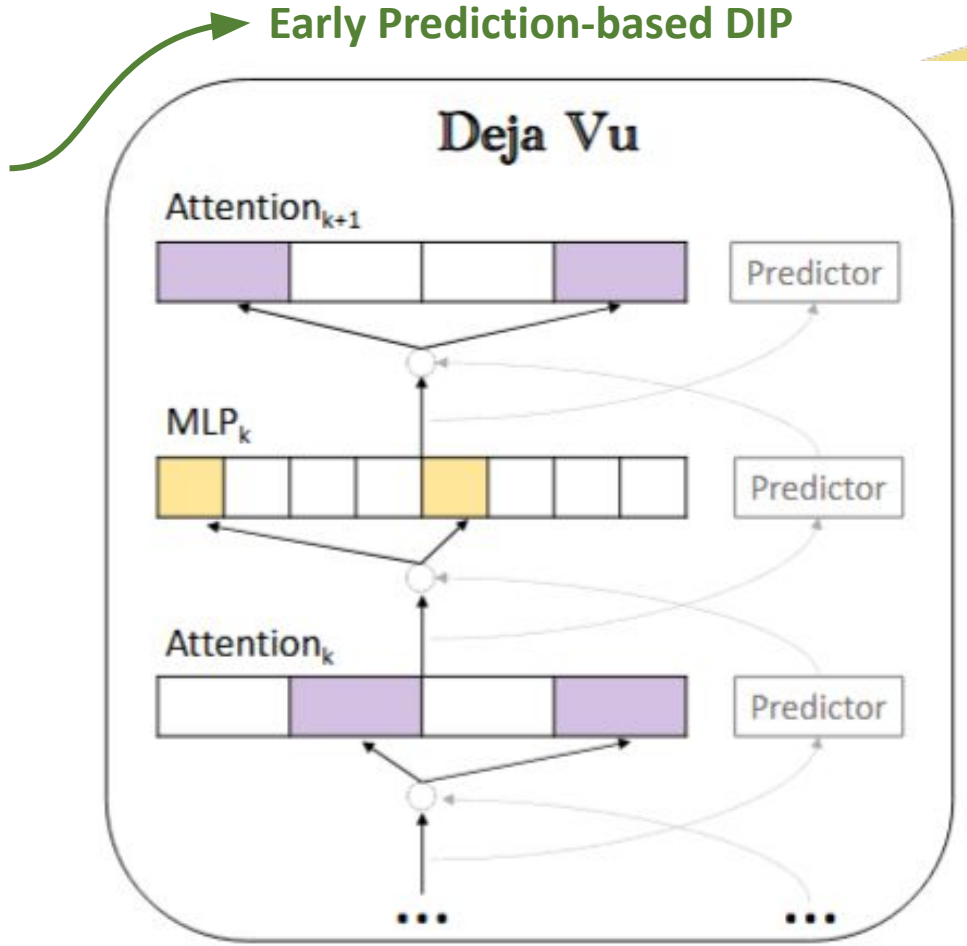
BASIC LAB

Traditional ReLU-based LLMs

- **High Natural Sparsity**: These models naturally produce a high degree of zero-valued activations.
- **Predictive Pruning**: Small predictors could accurately identify these zeros to enable effective weight pruning.
- **Stable Performance**: This inherent property allowed for acceleration without sacrificing accuracy.

Modern SwiGLU-based LLMs (e.g., Mistral-7B, Phi-3-Medium)

- **Low Natural Sparsity**: SwiGLU networks have very few "hard zeros," making them naturally dense.
- **Unpredictable Patterns**: While they can be pruned based on magnitude, the resulting sparsity patterns are difficult to forecast.
- **The Failure of Predictors**: Traditional predictor-based approaches become ineffective and cause significant accuracy degradation.

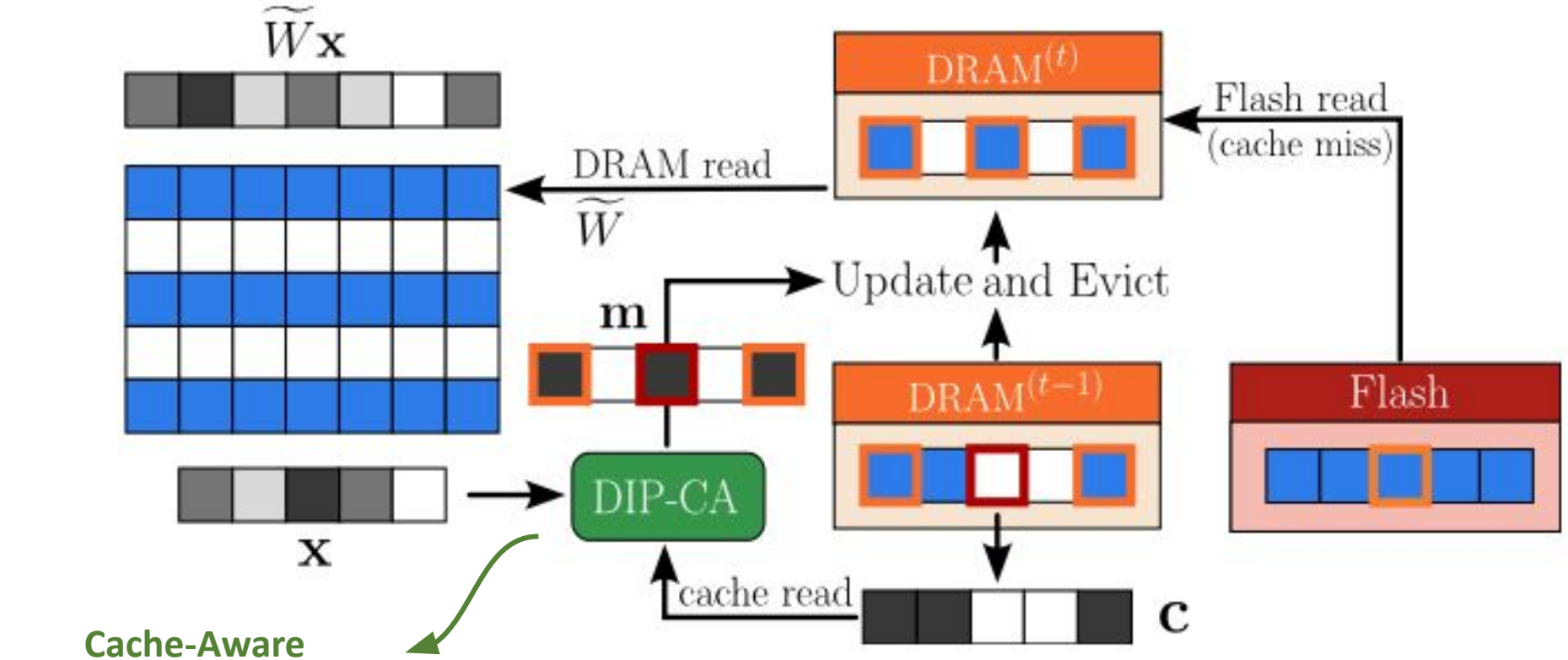


Efficient LLM Inference using Dynamic Input Pruning and Cache-Aware Masking

(MLSys 2025)

BASIC LAB

DIP-CA: select weights that have a significant impact on the output and are already in the cache, or weights that are extremely important and worth loading even if not cached.



Cache-Aware Dynamic Input Pruning module

Binary mask indicating which neurons are currently loaded into the DRAM cache.

NYCU

Dynamic Width Sparsity

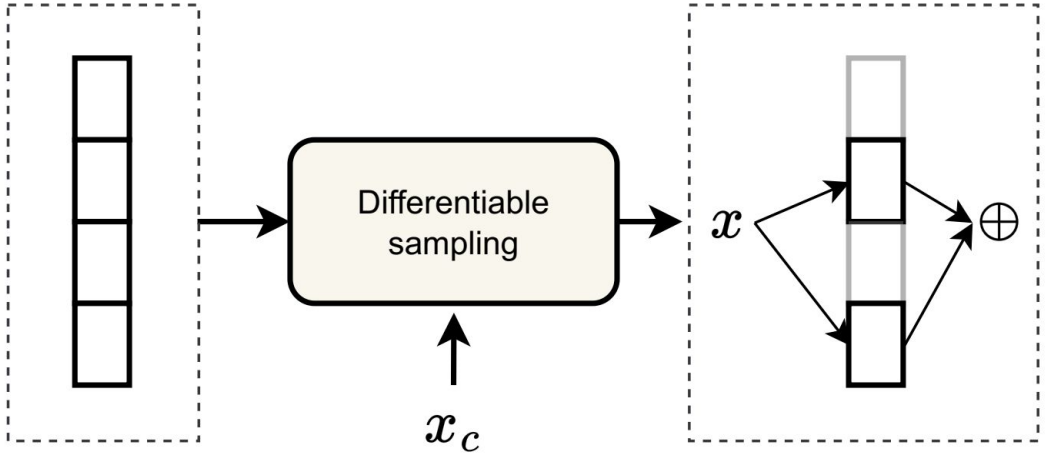


Figure 2: Width sparsity: Different parts of a layer (e.g., *experts*) can be activated based on the conditioning value.

Conditional computation in neural networks: principles and research trends
https://arxiv.org/abs/2403.07965

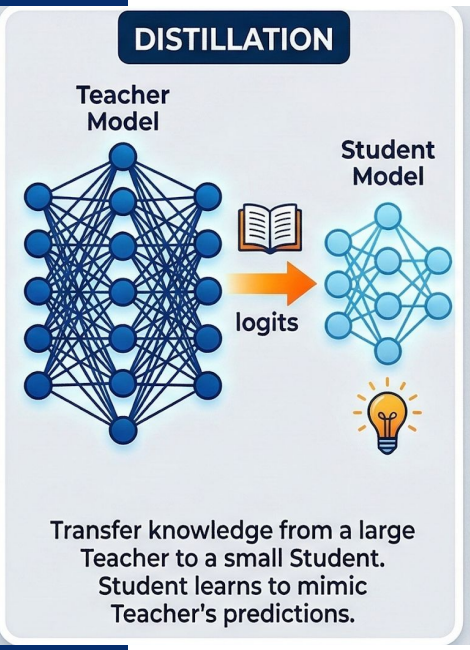
Dynamic Width Sparsity

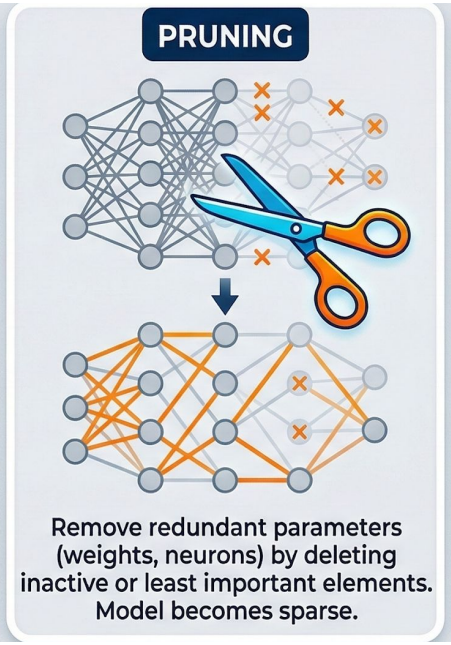
BASIC LAB

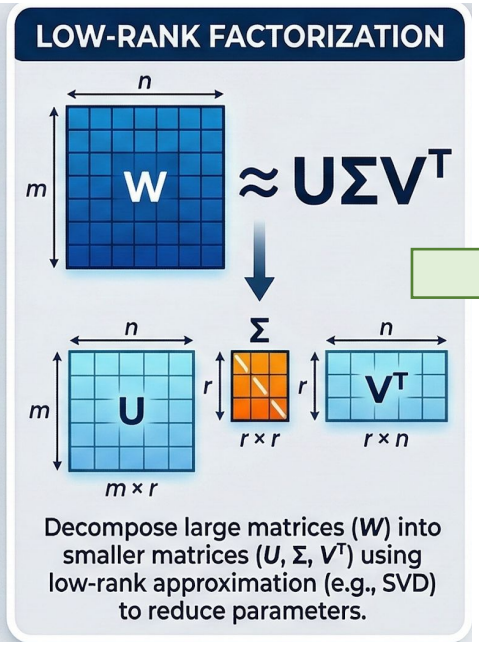
Traditional Approach

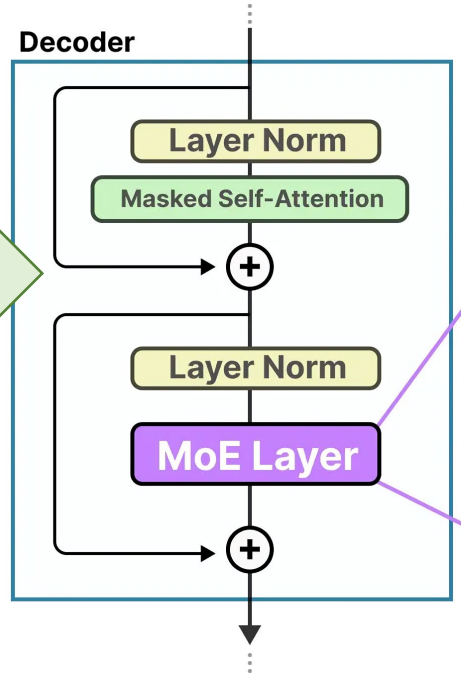
1. **Distillation**: Transferring knowledge from a large "teacher" to a compact "student" model.
2. **Pruning**: Permanently removing unimportant weights, neurons, or connections to reduce size.
3. **Low-rank Factorization**: Using lower-rank matrices to approximate original weights and reduce parameters.

The "Static" Trap: optimized before deployment; once finished, the computation graph remains fixed.

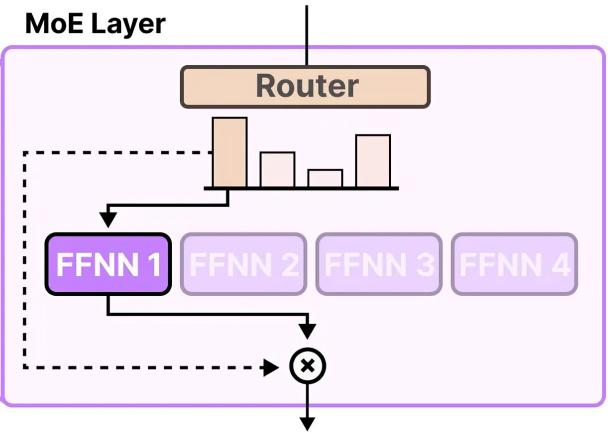






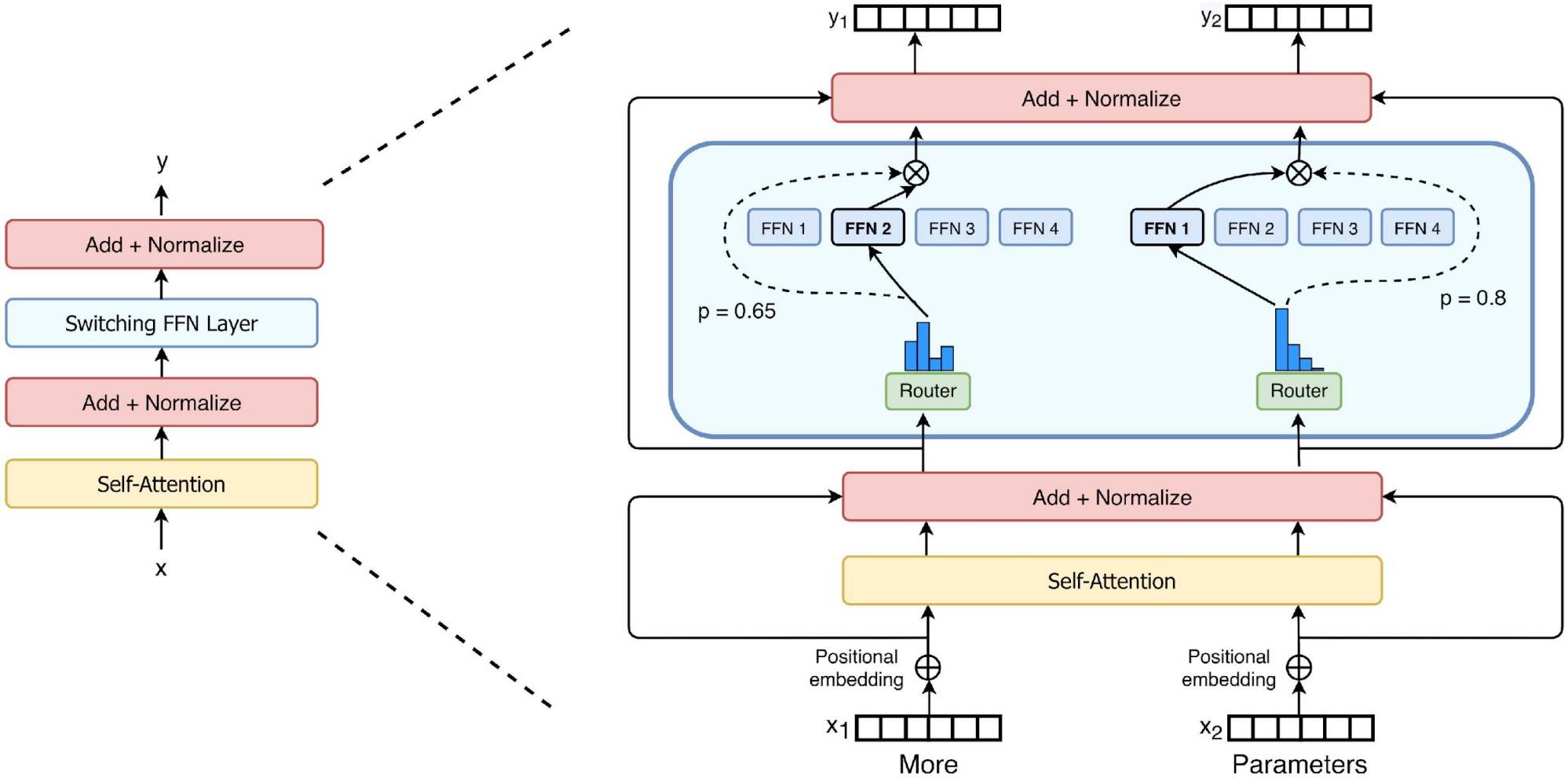


MoE Structure



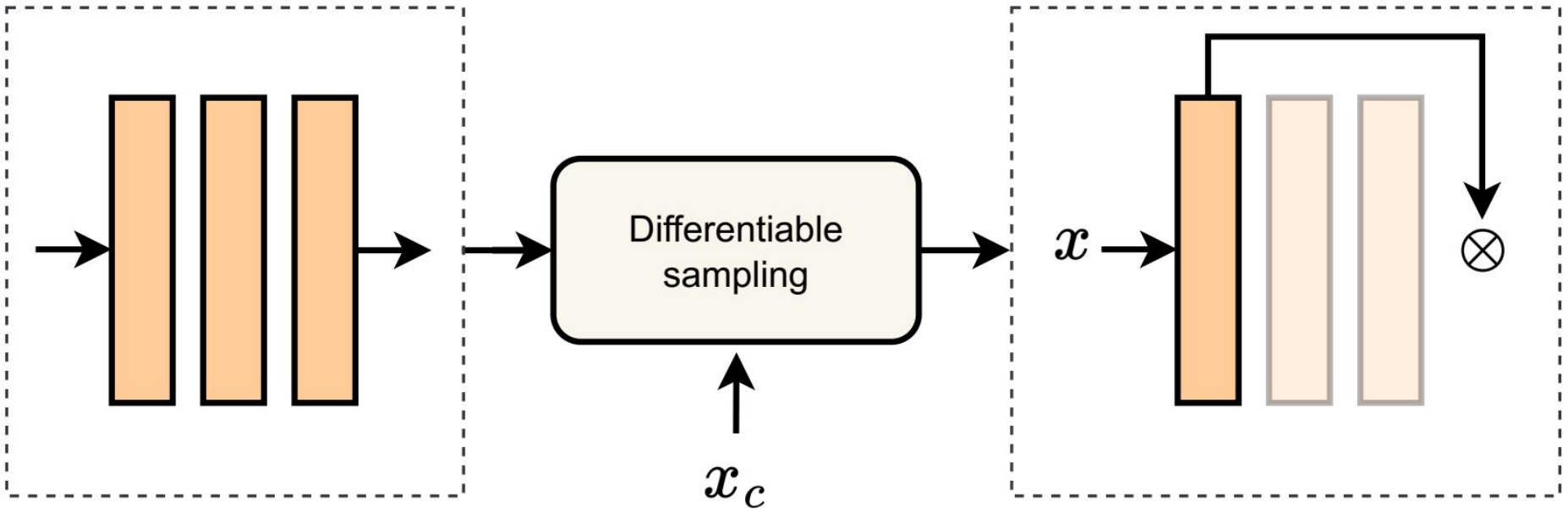
Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity

BASIC LAB



Dynamic Depth Sparsity

1. Pruning and Early Exit
2. Recurrence and Looped Architectures
3. Dynamic Routing and Modular Inference

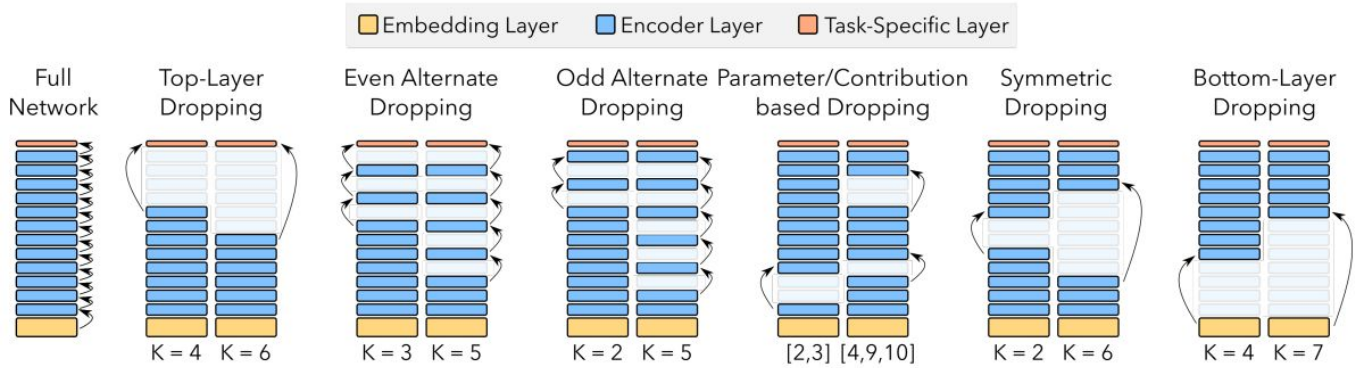


Conditional computation in neural networks: principles and research trends
https://arxiv.org/abs/2403.07965

1. Pruning and Early Exit

BASIC LAB

- **Pruning**: remove redundant weights, heads, layers.
- **Early exit**: attach auxiliary classifiers to intermediate layers, where an exit is triggered if the computed entropy falls below a predefined threshold, signifying sufficient confidence in the prediction.



On the Effect of Dropping Layers of Pre-trained Transformer Models

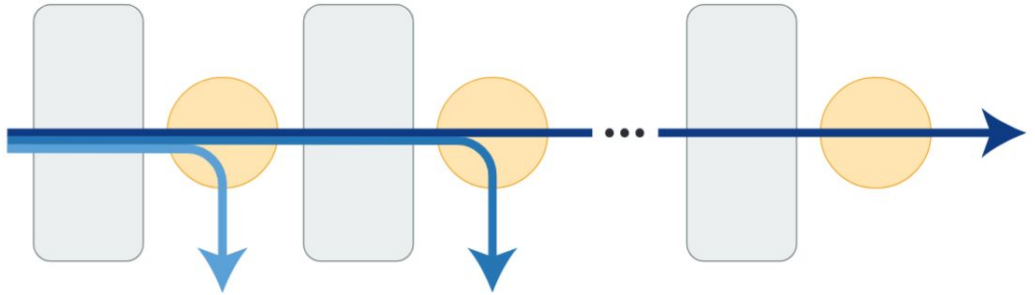
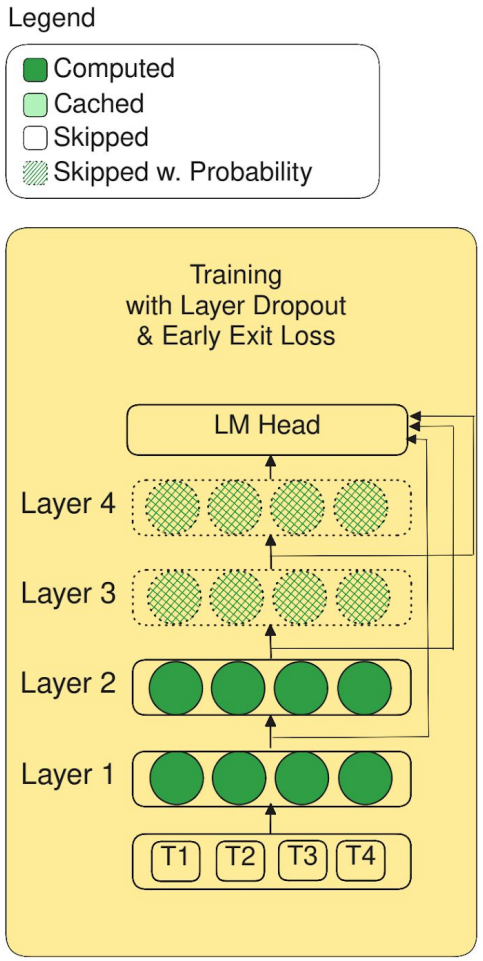


Figure 1: DeeBERT model overview. Grey blocks are transformer layers, orange circles are classification layers (off-ramps), and blue arrows represent inference samples exiting at different layers.

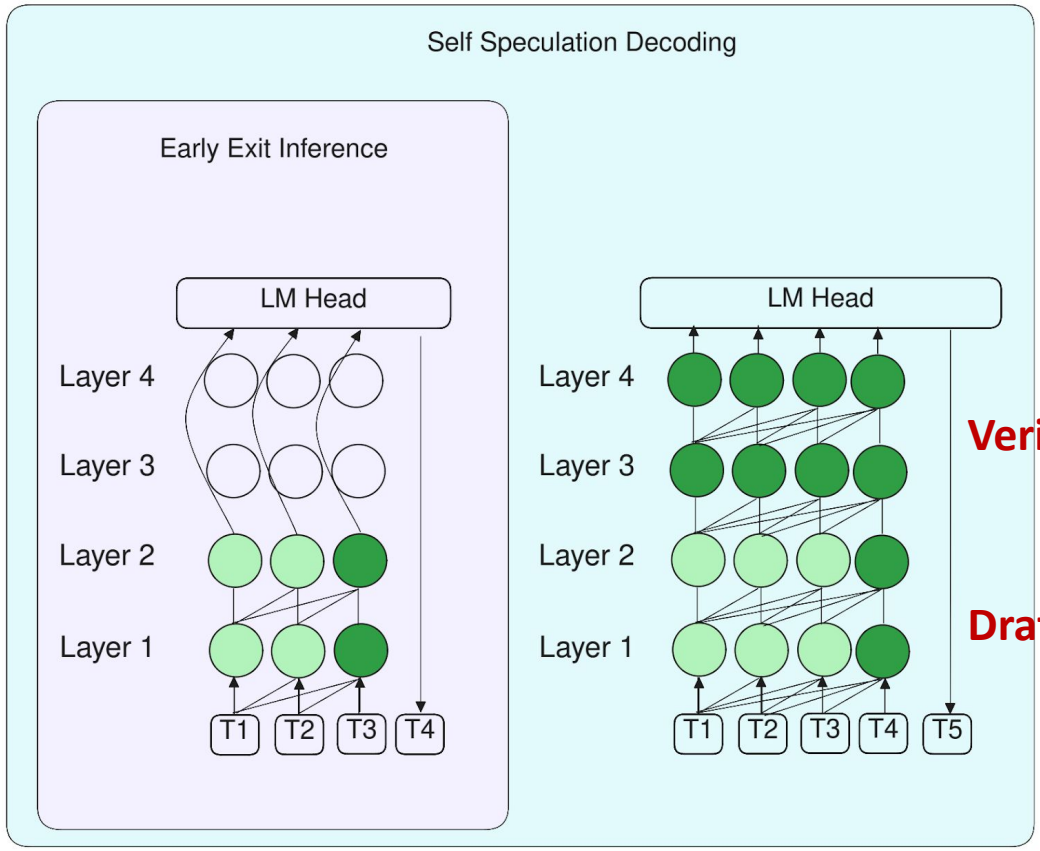
DeeBERT: Dynamic Early Exiting for Accelerating BERT Inference

LayerSkip: Enabling Early Exit Inference and Self-Speculative Decoding
(ACL 2024)

BASIC LAB



Train using Layer Dropout + Early Exit Loss....



... enables inference with subset of layers with higher accuracy...

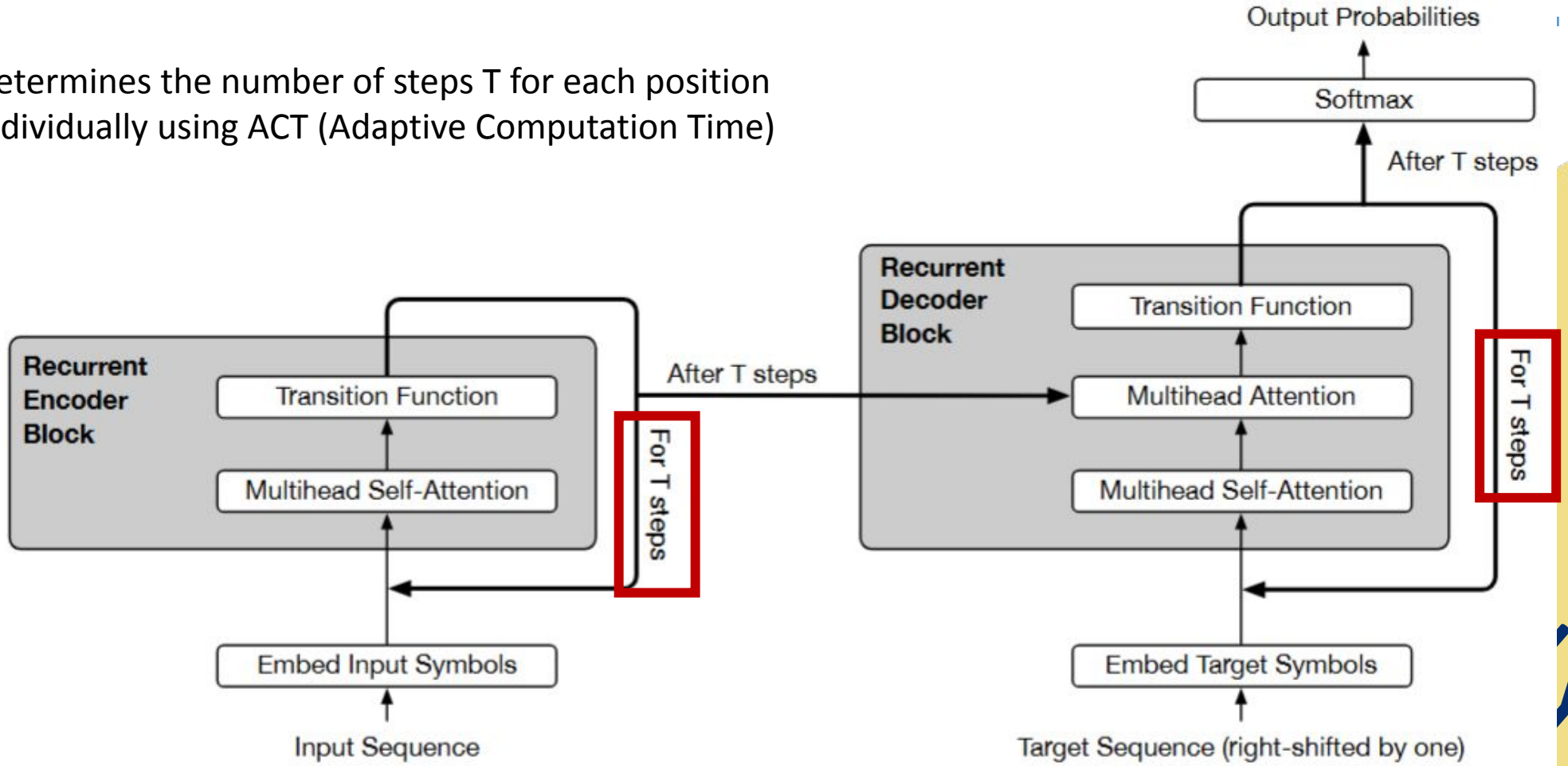
... and we can improve accuracy by verifying and correcting with remaining layers

Verification Stage

Draft Stage

2. Recurrence and Looped Architectures

- Determines the number of steps T for each position individually using ACT (Adaptive Computation Time)



Universal Transformers
https://arxiv.org/abs/1807.03819

3. Dynamic Routing and Modular Inference

BASIC LAB

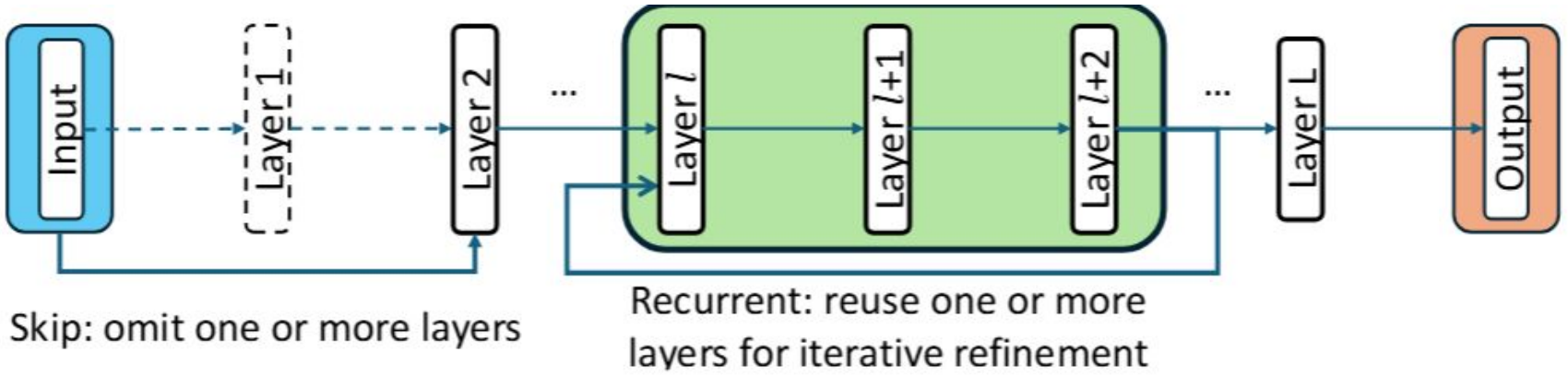
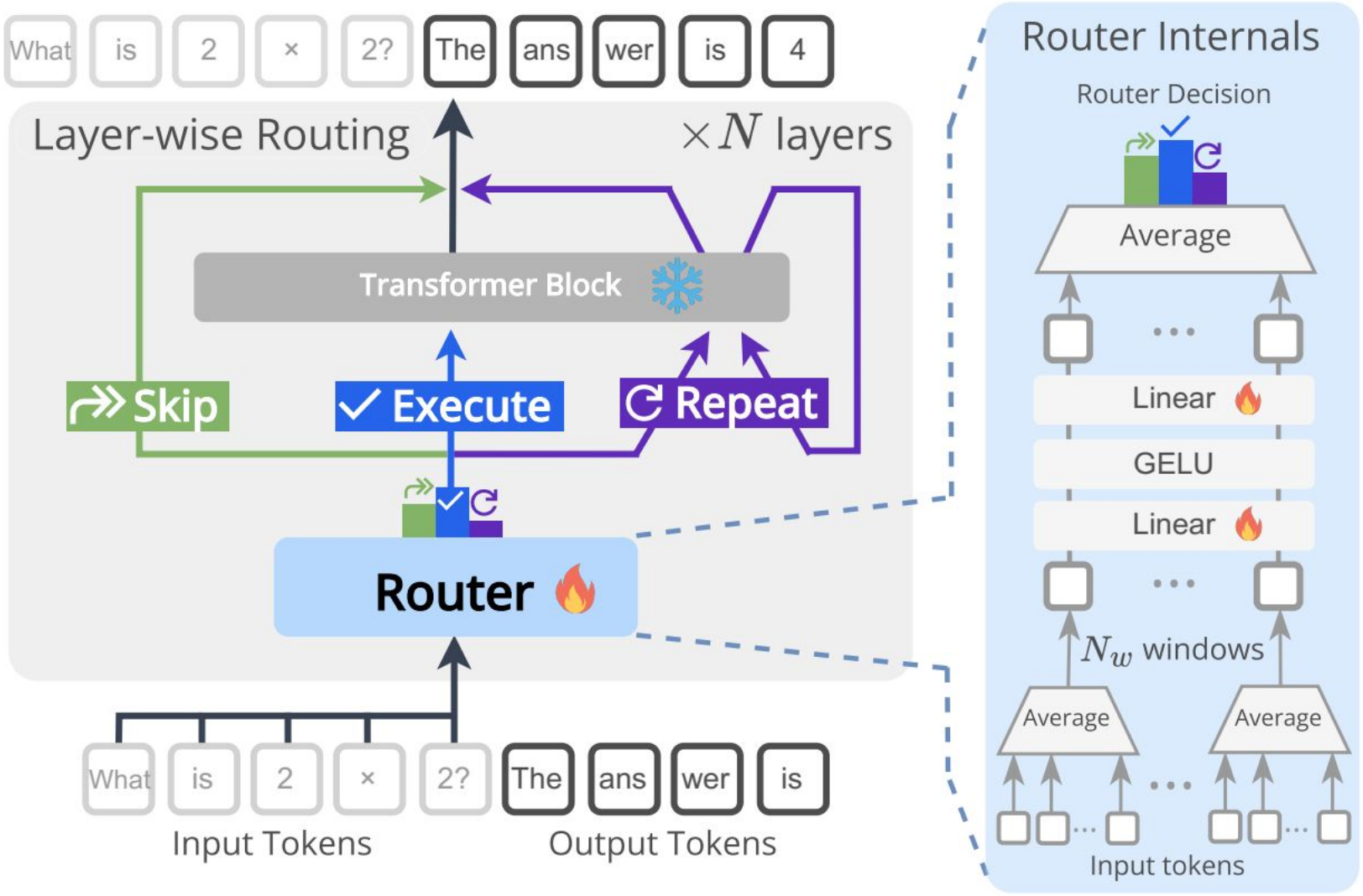


Figure 1: **Test-time layer composition search space** for CoLa. Starting from the original forward path, each input can dynamically skip or recurrently reuse any layer(s) to construct a custom Chain-of-Layers (CoLa). This joint space enables both layer pruning and recurrence, supporting fast-slow depth adaptation and dynamic architecture generalization from a pretrained model without any finetuning.

Skip a Layer or Loop it? Test-Time Depth Adaptation of Pretrained LLMs
https://arxiv.org/abs/2507.07996

Dr.LLM: Dynamic Layer Routing in LLMs
(ICLR 2026)

BASIC LAB



Windowed Mean Pooling for hidden state of input tokens

NYCU

Dr.LLM: Dynamic Layer Routing in LLMs
(ICLR 2026)

BASIC LAB

- Improving accuracy while simultaneously saving layers (i.e., reducing computation)

Method	Accuracy ↑	Retrofit	Cheap I	Cheap T	LLM ❄
CoLa	✓	✓	✗	✗	✓
Mixture of Depths	✗	✗	✓	✗	✗
Universal Transformer	✓	✗	✗	✗	✗
LLM-Pruner	✗	✓	✓	✗	✗
Mixture of Experts	✓	✗	✓	✗	✗
Mixture of Recursions	✓	✗	✗	✗	✗
LayerSkip	✗	✓	✓	✗	✓
ShortGPT	✗	✗	✓	✗	✗
MindSkip	✗	✓	✓	✗	✓
FlexiDepth	✗	✓	✗	✗	✓
Dr.LLM (Ours)	✓	✓	✓	✓	✓

Recurrence and Looped Architectures
Pruning & Early Exit
Dynamic Width Sparsity

NYCU

Dr.LLM: Dynamic Layer Routing in LLMs
(ICLR 2026)

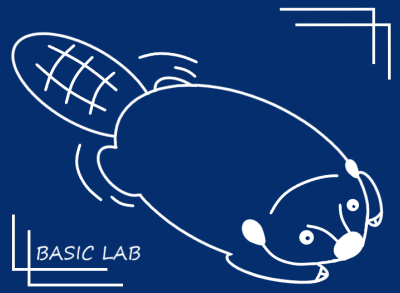
BASIC LAB

Limitations (from Hardware View)

- **Dynamic control flow**: Path varies for every prompt, making static compiler optimization and kernel fusion difficult.
- **Irregular Memory Access**:
  - Breaks cache prefetching and data locality when different tasks in a batch skip different layers.
  - Leads to suboptimal throughput on vectorized hardware (GPUs/TPUs).
- **Execution Overhead**: The cumulative latency of Router MLPs and Windowed Pooling can offset the time saved from skipping layers if not carefully implemented.
- **Per-Prompt Constraints**: Requires sequence-level consistency to maintain KV-cache efficiency

NYCU

Proposed Research



BASIC LAB

NYCU

Proposed Research 1: Hardware-Friendly Highway Strategy

BASIC LAB

- **Problem of Dynamic Routing:**
  - Input-dependent paths cause Warp Divergence and irregular memory access on GPUs.
  - Frequent jumps lead to Instruction Pipeline flushes and Cache Misses.
- **Goal:** Efficient dynamic routing (Dr. LLM) with hardware-friendly highway.
- **Defining "Hot Highways":**
  - Identifying frequent contiguous layer combinations (e.g., Block 1-2-3).
  - Converting unpredictable jumps into **Input-Agnostic Execution Paths** at compile-time

NYCU

Proposed Research: Hardware-Friendly Highway Strategy

BASIC LAB

Step 1. Stability-Stability-Aware Structure Mining

- Four Metrics for Highway Discovery:
  - **Support**: Frequency of a specific routing pattern (e.g., Skip Layer 7-9) in the training set.
  - **Skip Variance**: Quantifying cross-prompt instability; low variance indicates a candidate for compile-time fixation.
  - **Accuracy Drop**: Measuring the performance impact of forcing a dynamic layer into a static state.
  - **Hardware Contiguity**: Identifying continuous blocks (e.g., 1-2-3) suitable for Operator Fusion.

NYCU

Proposed Research:
Hardware-Friendly Highway Strategy

BASIC LAB

Step 2. Compiler & Hardware Mapping Pipeline

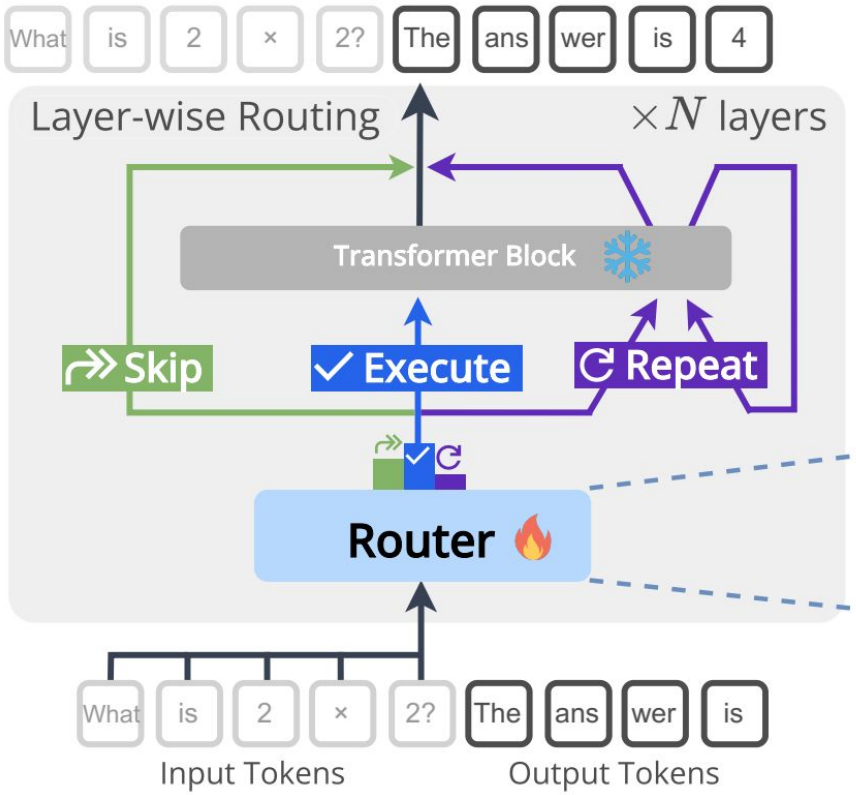
- **Path Profilling:** Extracting path locality for specific tasks.
- **Graph Transformation:** Converting conditional branches into specialized **"Fast-path" Sub-graphs**.
- **Kernel Fusion:** Merging consecutive Highway blocks into a single CUDA/Triton kernel to minimize VRAM I/O.
- **Static Path Optimization:** Establishing dedicated physical **Data Paths** (in FPGA/ASIC) for high-frequency routes to minimize unit-to-unit latency.
- **Block Clustering:** Mapping frequently co-executed blocks onto **physically adjacent Processing Elements (PEs)** to maximize local SRAM reuse.
- **Static SRAM Allocation:** Pre-loading Highway weights/activations into SRAM to eliminate dynamic scheduling bubbles.

NYCU

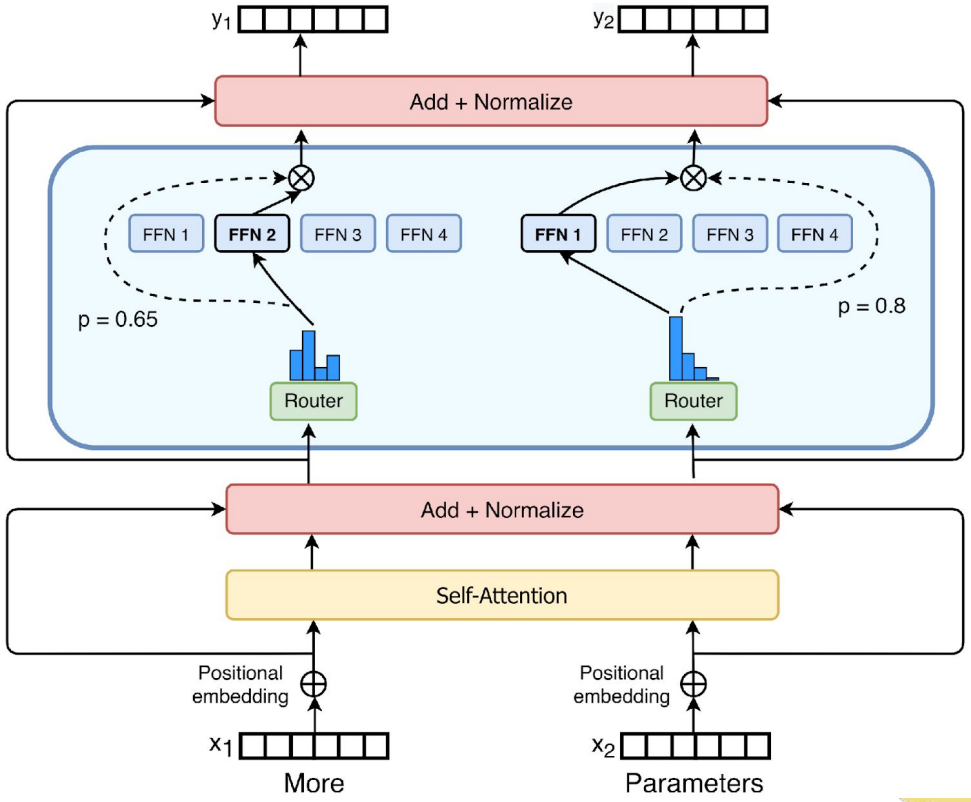
Extension to MoE Structure

BASIC LAB

Layer-wise Routing

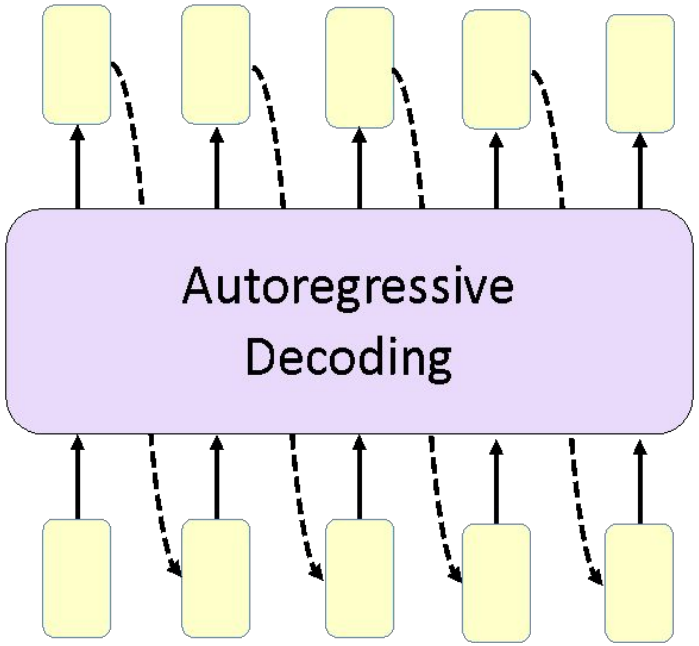


Expert-wise Routing

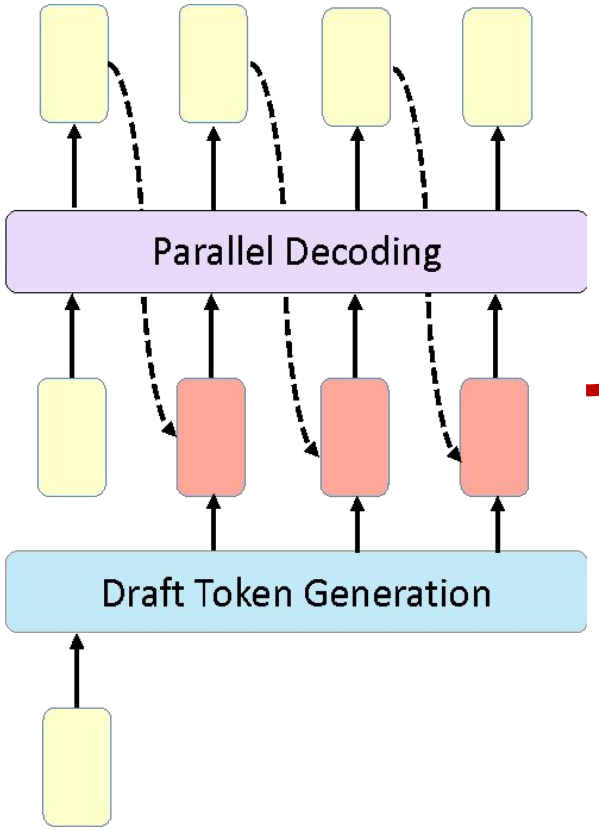


Proposed Research 2:
MoE Speculative Decoding

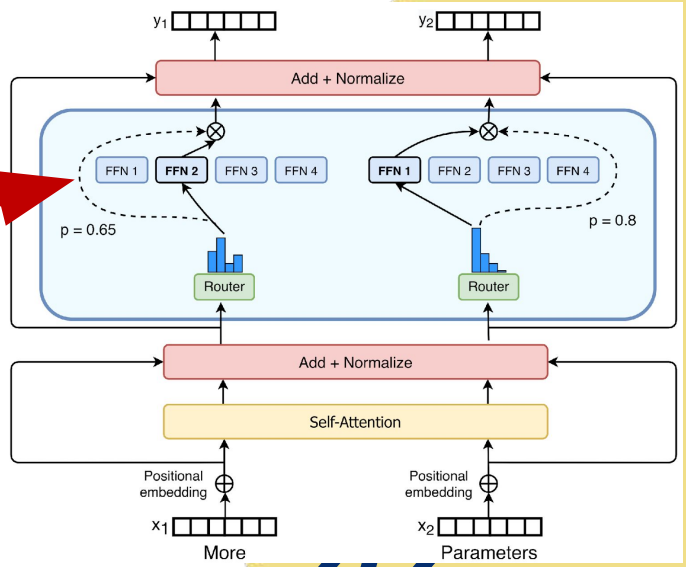
BASIC LAB



(a)

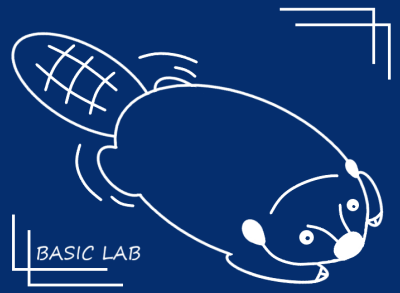


(b)



THANK YOU FOR LISTENING

Big data Analytics and Social Intelligent Computing



BASIC LAB

NYCU